

第 2 次课实验指导书：.data/.bss 初始化与 C/汇编混合启动

技术编号：02 | 建议检查点：02-data-bss
称“第 2 次课”，不称“第 2 周”。配合讲义：docs/02/lecture_notes.md。
环境总手册（强烈建议收藏）：docs/student_env_runbook.md

1. 实验目标

完成本次课最小可验证闭环，主题：**.data/.bss 初始化与 C/汇编混合启动**。

预期/产出：

```
Hello miniOS on LoongArch64
data section ok
bss section cleared
week1-week2 check done
```

text

2. 实验环境与准备（必须先做对，否则后面全错）

2.0 课程默认前提

- 第 1 周已完成：本机装好 WSL + Ubuntu。本指导书从「进入 Ubuntu」写起，不重复安装步骤。
- 所有 make / 编译 / QEMU 只在 Ubuntu 里做。
- 总手册：docs/student_env_runbook.md。

2.1 命令必须在哪里敲？

窗口提示符	你在哪	能不能敲 make
PS D:\...>	Windows PowerShell（还没进 Ubuntu）	不能
user@xxx:~\$	已进入 Ubuntu	能

口诀：有 PS 前缀 = 还在 Windows = 先进入 Ubuntu。

PowerShell 里若出现 无法将“make”项识别为 cmdlet / ObjectNotFound: make :
说明没进 Ubuntu 就敲了 make，按 §2.2 重做。

2.2 标准操作顺序：先从 WSL 进入 Ubuntu，再 make

步骤 1：从 WSL 进入 Ubuntu

在 **PowerShell**（此时是 `PS ...>`）中输入：

```
ws1 -d Ubuntu
```

powershell

或从开始菜单打开 **Ubuntu**。

成功标志：提示符变成 `用户名@主机名:~$`（没有 `PS`）。

若失败：在 PowerShell 执行 `ws1 -l -v`，确认有 Ubuntu，再用 `ws1 -d <NAME>`。

步骤 2：在 Ubuntu 里进入仓库

```
cd "/mnt/d/日常教学/2026-2027第一学期/汇编语言/miniOS"  
pwd  
ls Makefile
```

（`D:\foo` → `/mnt/d/foo`，按你本机路径改。）

步骤 3：工具自检（第 1 次课装过可快速确认）

```
which make && which loongarch64-linux-gnu-gcc && which qemu-system-loongarch64
```

缺则 `sudo apt install -y make qemu-system-misc gcc-loongarch64-linux-gnu binutils-loongarch64-linux-gnu gdb-multiarch`。

步骤 4：此后 git / make 都在本 Ubuntu 窗口

```
pwd  
git status
```

2.2 重点阅读文件

- `boot/start.S`
- `kernel/main.c`
- `kernel/linker.ld`
- `lib/string.S`

- `include/string.h`

2.3 关于 `my-02-lab` 分支（必读）

名称	是什么	在 GitHub / <code>origin</code> 上?
<code>02-data-bss</code>	课程 tag （第 2 次课验收快照）	有
<code>master</code>	已发布最新课次纯净代码	有
<code>my-02-lab</code>	你的本地实验分支	默认没有

- 远程没有 `my-02-lab` 完全正常。
- 第 2 次课请**另建** `my-02-lab`，不要在 `my-01-lab` 上混做。
- 详解：`docs/01/student_git_tag_guide.md`（通用原则）、`docs/02/student_git_tag_guide.md`（本次课命令逐条解释 + 踩坑记录）。

2.4 课次代码范围（纯净性）

本 tag 包含：`clear_bss`、`.data/.bss` 验证、最小 `string.S`。
本 tag 不包含：异常入口、系统调用、中断等后续课内容。

3. 实验任务（全部在 Ubuntu 仓库根目录完成）

Task1 检出代码（创建本地实验分支）

```
git fetch --tags
git switch -c my-02-lab 02-data-bss
git branch
git tag | grep 02
```

自检：

- 当前分支为 `* my-02-lab`（或你自定名）
- 存在 tag `02-data-bss`
- **不要**期望远程有 `origin/my-02-lab`

若分支已存在：`git switch my-02-lab`，或
`git switch -c my-02-lab-2 02-data-bss`。

Task2 编译运行

完整顺序回顾（抄一遍）：

```
wsl -d Ubuntu
```

powershell

进入 Ubuntu 且 `cd` 到仓库、分支正确之后：

```
make clean
make
make run
```

禁止在仍显示 `PS ...>` 的窗口里敲 `make`。

预期四行：

```
Hello miniOS on LoongArch64
data section ok
bss section cleared
week1-week2 check done
```

text

退出 QEMU：**Ctrl+a**，再按 **x**。

Task3 段与变量

- 指出 `data_message` 与 `bss_buffer` 分别更可能落在哪一类段。
- 说明 `.text/.rodata/.data/.bss` 各放什么。

Task4 精读 `clear_bss`

- 逐行注释 `clear_bss`：边界、循环条件、`st.b`、指针递增、返回。
- 说明 `__bss_start` / `__bss_end` 来自何处。

Task5 混合调用

- 从 `main.c` 找到 `memset/memcpy/strlen` 调用点。
- 对照 `string.h` 声明与 `string.S` 实现，写出参数寄存器对应（可简表）。

4. 验收标准

- 四行输出齐全且与 tag 预期一致
- 能解释为何进 C 前要清 `.bss`

- 报告含真实命令输出
- 能说明： `make` 必须在 WSL/Linux 中执行，不能在 PowerShell 中执行

5. 实验报告要求

报告至少包含：

1. 环境说明（OS、是否 WSL、工具版本摘要）
2. 关键命令与**真实输出**（可截断，但不可编造）
3. 对核心代码/指令的简要注释或流程图
4. 问题与解决过程（若有）
5. 思考题作答

6. AI 共学边界

允许解释 `clear_bss` 与段含义；实验结果必须来自真实命令输出。

7. 思考题

1. 为何 `.bss` 通常不在镜像里存满 0？
2. 不清 `.bss` 时 `bss_buffer` 可能怎样？
3. `strlen` 遇到无 `\0` 结尾会怎样？

8. 提交清单

- ☐ 实验报告（PDF/Markdown）
- ☐ 关键输出摘录
- ☐ 需要提交的代码补丁或笔记（按教师要求）

9. 常见故障速查

现象	原因	处理
<code>ObjectNotFound: make</code> / 无法将 <code>make</code> 项识别为...	在 PowerShell 敲了 <code>make</code>	<code>wsl -d Ubuntu</code> → <code>cd 仓库</code> → 再 <code>make</code>

现象	原因	处理
只有 Hello、没有 data/bss 行	仍在 01 树或未清编译	确认 tag/分支为 02， <code>make clean && make && make run</code>
缺 <code>bss section cleared</code>	<code>clear_bss</code> 未执行或验证被改	对照 <code>start.S</code> 是否在 <code>bl kernel_main</code> 前 <code>bl clear_bss</code>
<code>undefined reference to memset</code>	未链接 <code>lib/string.S</code>	检查 <code>Makefile</code> 的 <code>SRCS_S</code> 含 <code>lib/string.S</code>
退不出 QEMU	退出键不对	Ctrl+a 然后 x